

월 5,000명이 쓰는 서비스를 전면 재설계해 운영하고,
다양한 직군과 협업하며 실제 서비스를 만들어 온 개발자입니다.

이런 경험을 했습니다

기술 전환, 비용 절감

레거시를 새 스택으로 다시 개발하고 월 서버비를 80만원에서 25만원으로 줄였습니다.

운영 중 장애 해결

결제 후 출력이 멈추는 장애의 원인을 찾아 통신 구조를 바꿨습니다.

협업, 서비스 운영

6인 팀으로 앱을 출시하고 매주 회의하며 함께 운영합니다.

공드림

대학 무인 프린트 서비스
2025.12 ~ 운영 중, 전 영역 단독 개발
서버비 1/3 절감, 오류 감소

문제 해결 미리보기

- 웹소켓 zombie 커넥션을 폴링으로 단순화
- 로그인 확인 서버를 JWT로 제거
- 먹등키와 Lock으로 결제 보호

커머스

선착순 이벤트를 가정한 커머스 백엔드
2026.01 ~ 2026.03, 개인 프로젝트 1인 개발
Java, Spring Boot, Redis, MySQL

문제 해결 미리보기

- Redis Lua로 쿠폰 발급을 원자 처리
- 재고는 비관적 락, 잠금 순서 고정
- 대기열과 세션 자동 복구 설계

FryDay

TODO 서비스
2025.11 ~ 운영 중, 백엔드 2인 중 1
푸시 안정화, 협업 경험

문제 해결 미리보기

- 낙관적 락으로 알림 동시 수정 제어
- 트랜잭션 분리로 중복 발송 차단
- 리포트 즉시 계산과 비동기 조회

학력 · 교육

공주대학교 소프트웨어학과 (2027년 2월 졸업예정)
SKALA - SK AI Leader Academy (2026.07 ~ 현재)

자격증

정보처리산업기사 (2024.12)
SQLD (2024.04)

공드림

외주로 만들어진 대학 무인 출력 서비스를 넘겨받아
서버부터 앱까지 전 영역을 다시 만들고 운영하고 있습니다.

2025.12 ~ 운영 중 · 전 영역 단독 개발 · Java, Spring Boot, MySQL, NCP, Docker

- 01 넘겨받은 시스템의 문제와 인프라 재설계
- 02 즉시 통신을 위한 웹소켓의 한계와 폴링 전환
- 03 로그인 확인 서버 제거와 비용 절감
- 04 QR 동시 스캔에서의 결제 정합성
- 05 사용자 증가에 대비한 확장 설계

오류 찾고, 비싸고, 데이터도 못 믿던 시스템을 혼자 처음부터 다시 만들었습니다

2025.12 ~ 운영 중 · 전 영역 단독 개발 · Java, Spring Boot, MySQL, NCP, Docker

공드림은 대학 무인 프린트 서비스입니다. 손님이 앱에서 문서를 보내면 매장 프린터로 출력되고 그 자리에서 결제가 일어납니다. 지인이 외주로 개발한 기존 시스템을 넘겨받아 서버부터 매장 프로그램, 앱, 관리자 화면까지 전부 다시 만들어 운영하고 있습니다.

넘겨받은 시스템의 문제

오류

작은 오류가 잦는데 오류를 기록하고 복구하는 장치가 없었습니다. 동시 요청을 막는 락 같은 방어도 없었습니다.

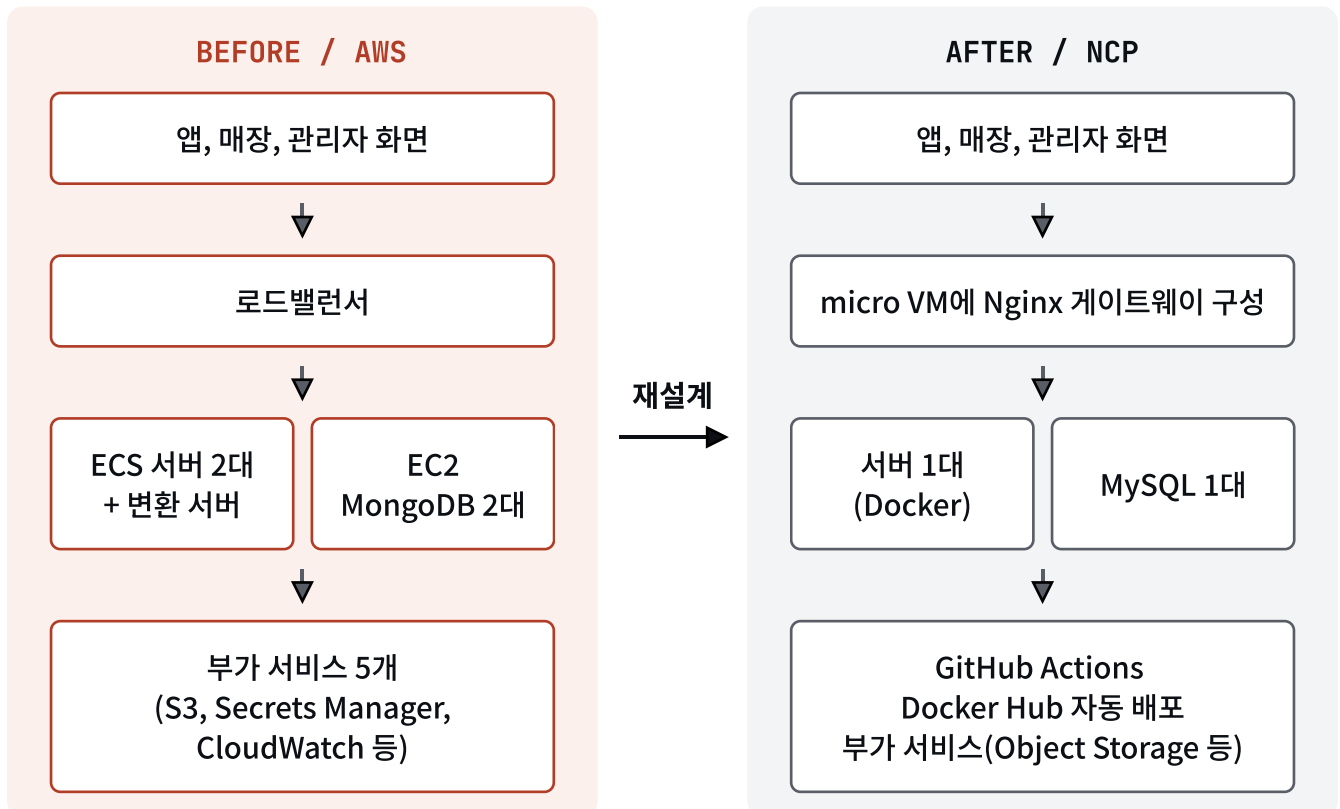
비용

작은 서비스인데 서버가 여러 대로 쪼개져 월 80만원이 서버비로 청구 되었습니다.

데이터

2만 명 가입자에 이메일 인증도 동의 기록도 없었습니다. 탈퇴 회원에 대한 처리도 존재하지 않았습니다.

Infra를 이렇게 바꿨습니다



로드밸런서, ECS 컨테이너, EC2 위 DB로 흩어져 있던 인프라를 작은 VM의 Nginx 게이트웨이와 컨테이너 서버, MySQL로 줄였습니다. 회원 데이터는 버리고 소셜로그인 또는 이메일 인증과 개인정보 동의를 제대로 받는 구조로 다시 모았습니다.

80 > 25만원

월 서버비

5대 > 3대

관리 서버 및 인프라

월 활성 5,000명

재설계 후 실사용자

일 활성 861명

매일 쓰는 사용자

즉시 통신을 위해 도입한 웹소켓의 한계를 겪고 통신을 폴링 하나로 단순화했습니다

문제

현장출력과 원격출력 모두 지연 없이 처리하려고 웹소켓을 도입했습니다. 그런데 매장이 있는 학교 네트워크는 가끔 끊기고 통신 오류도 잦았습니다. 데스크탑 앱과 서버 사이 연결이 실제로는 끊겼는데 정상처럼 보이는 좀비 커넥션이 자주 생겼고 결제된 문서가 재부팅 전까지 출력되지 않았습니다.

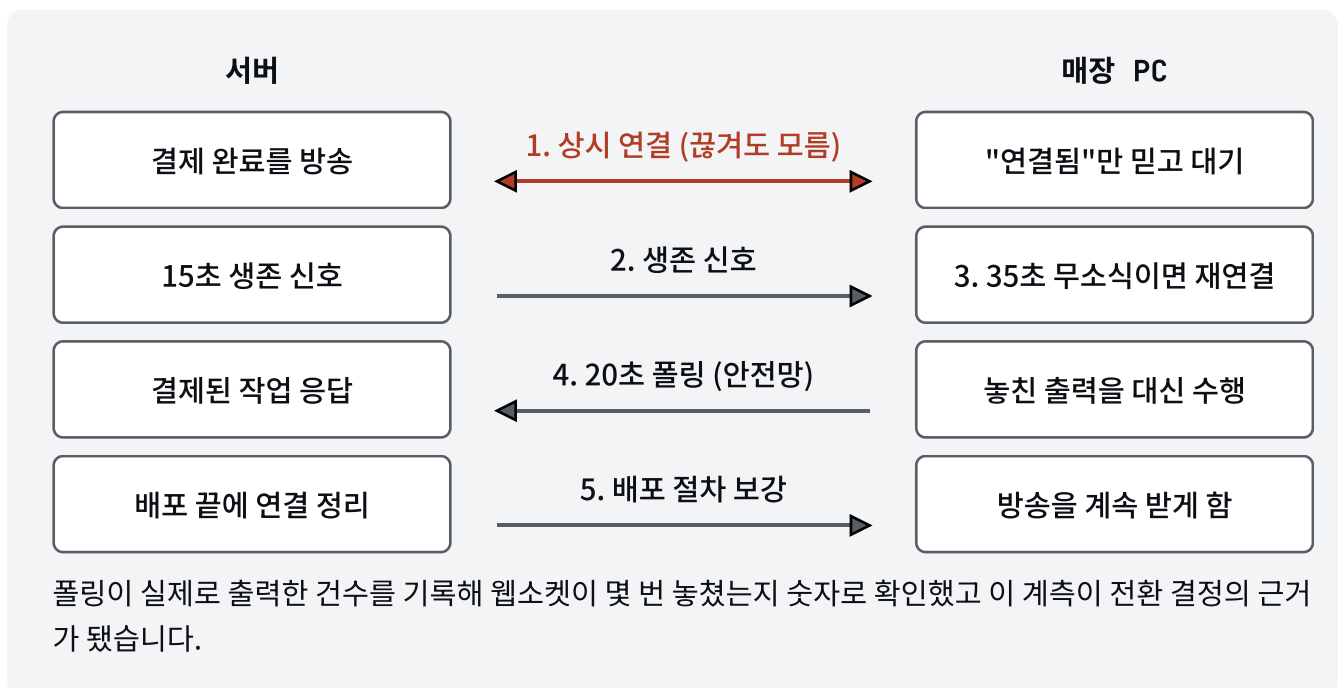
해결

끊김을 감지하려고 생존 신호와 위치독을 붙였습니다. 농친 결제를 되살리려고 20초 폴링을 더했고 중복 출력을 막으려고 멍등성 처리를 위해 출력 기록을 남겼습니다. 문제가 보일 때마다 방어 장치를 하나씩 추가한 것입니다.

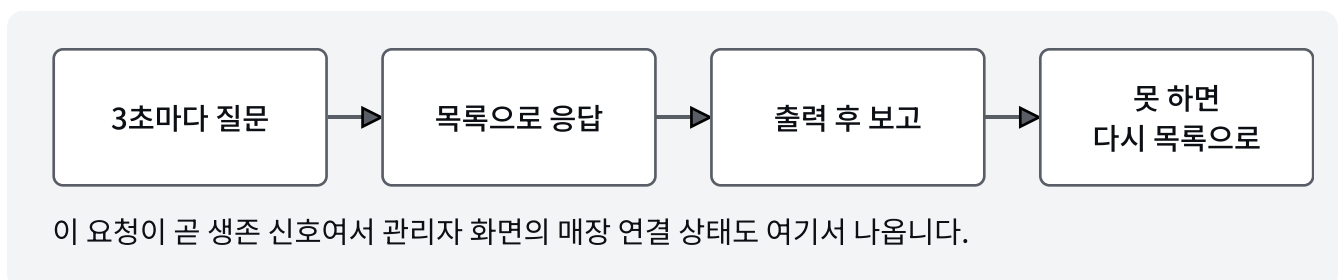
전환

그렇게 웹소켓 하나를 지키는 장치가 다섯 개까지 늘었습니다. 반면 폴링은 지연이 3초 정도로 충분히 짧았습니다. 몇 초의 지연과 유지 비용을 저울질해 폴링으로 바꿨습니다.

웹소켓 방식. 하나를 지키던 장치 다섯 개



폴링 방식. 하나면 충분합니다



5개 > 1개

지켜야 할 장치

0개

매장 상시 연결

3초

결제 확인 폴링 주기

모든 요청이 거치던 로그인 확인 서버를 필요 없게 만들고 서버 비용을 줄였습니다.

문제

기존 시스템은 로그인 확인만 하는 서버가 따로 있어 사용자의 모든 요청이 서버 두 대와 DB 두 개를 거쳤습니다. 응답은 느려지고 그 서버와 DB는 요청이 없어도 24시간 떠서 비용만 들어갔습니다.

해결

로그인을 JWT 토큰 방식으로 바꿨습니다. 토큰에 서명이 들어 있어 서버가 그 자리에서 진위를 확인할 수 있고 확인 전용 서버와 DB 자체가 필요 없어집니다. 그 덕분에 남은 기능을 서버 한 대로 합칠 수 있었습니다.

결과

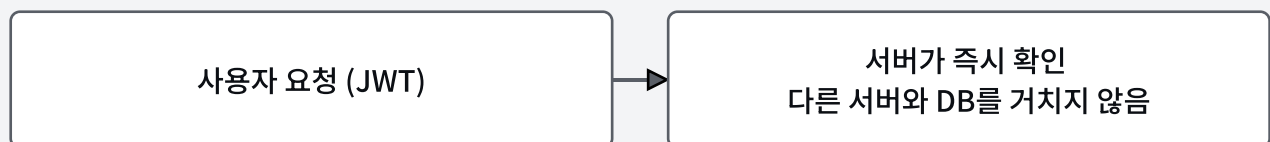
요청당 서버 왕복 0번. 확인 서버와 전용 DB를 제거했고, 이것이 앞 장의 서버 통합과 비용 절감을 가능하게 한 열쇠였습니다. 코드에 적혀 있던 비밀키도 전부 환경변수로 옮겼습니다.

로그인 확인이 거치는 길

BEFORE, 모든 요청이 이 길을 지나갔습니다



AFTER, 서버가 그 자리에서 끝냅니다



회원 데이터, 처음부터 제대로



인증해야 가입되고 약관과 개인정보 동의는 시각까지 저장됩니다. DB와 코드가 어긋나면 서버가 뜨지 않고 탈퇴한 계정은 보관 기간이 지나면 자동으로 지워집니다.

왕복 0번

로그인 확인 통신

비밀키 0개

코드에 남아 있던 키

실제 MySQL로 검증

H2 대신 컨테이너 사용

여러 폰이 같은 QR을 찍어도 결제는 한 번만 되도록 만들었습니다

문제

현장 출력은 화면에 뜬 QR을 폰으로 찍어 결제합니다. 한 QR을 여러 폰이 찍거나 결제 화면에서 뒤로 갔다 다시 들어오면 같은 출력 건에 결제 요청이 겹쳐 들어옵니다.

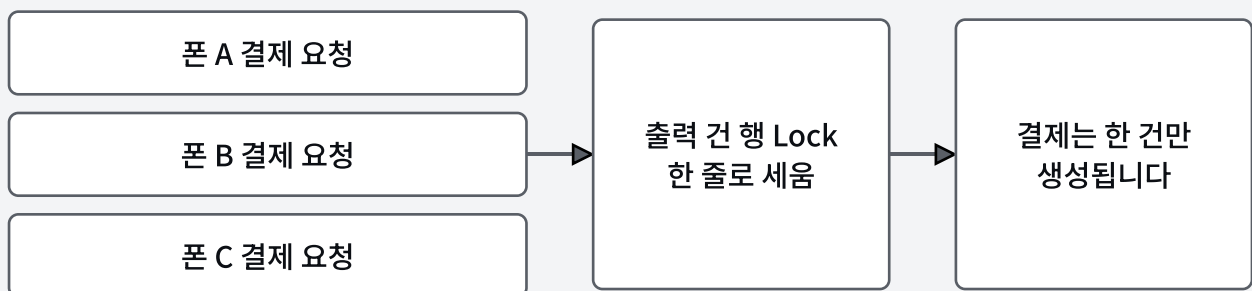
Lock

결제를 시작할 때 그 출력 건의 행을 비관적 락(Pessimistic Lock)으로 잠급니다. 동시에 들어온 요청은 한 줄로 세워지고 먼저 온 요청이 끝나야 다음 요청이 상태를 읽습니다.

역등성

출력 건과 결제수단으로 역등키를 만들고 등록하기 전에 같은 키의 결제를 먼저 찾습니다. 있으면 새로 만들지 않고 처음 만든 결제를 그대로 돌려주며 유일성은 DB 유니크 제약이 보장합니다.

같은 QR을 여러 폰이 찍었을 때



세 대가 같은 순간에 눌러도 행 Lock이 요청을 한 줄로 세우고 역등키가 그중 한 건만 남깁니다. 유일성 검사를 코드가 아니라 DB 유니크 제약에 맡긴 것이 핵심입니다.

두 번째 요청에 무엇을 돌려줄지



처음에는 유니크 제약이 막아주니 충분하다고 봤습니다. 그런데 막힌 요청이 오류로 나가면 화면은 알 수 없는 오류를 띄우고 손님은 다시 누릅니다. 막는 대신 처음 만든 결제를 그대로 돌려주도록 바꿨습니다.

중복 결제 0건

설계상 불가, 사고 없음

요청은 한 줄로

출력 건 행 비관적 락

소유자 대조

남의 결제를 못 열게

사용자가 늘면 서버만 늘리면 되도록 인프라와 코드를 확장 가능하게 설계했습니다

배경

재런칭 뒤 5,000명이 가입했고 개강 후에는 하루 평균 200명씩 새로 들어옵니다. 지금은 매일 500명에서 1,000명이 쓰는데 서버는 한 대로 여유가 있습니다. 다만 늘릴 때마다 코드를 고쳐야 한다면 확장이 아니라 공사가 됩니다.

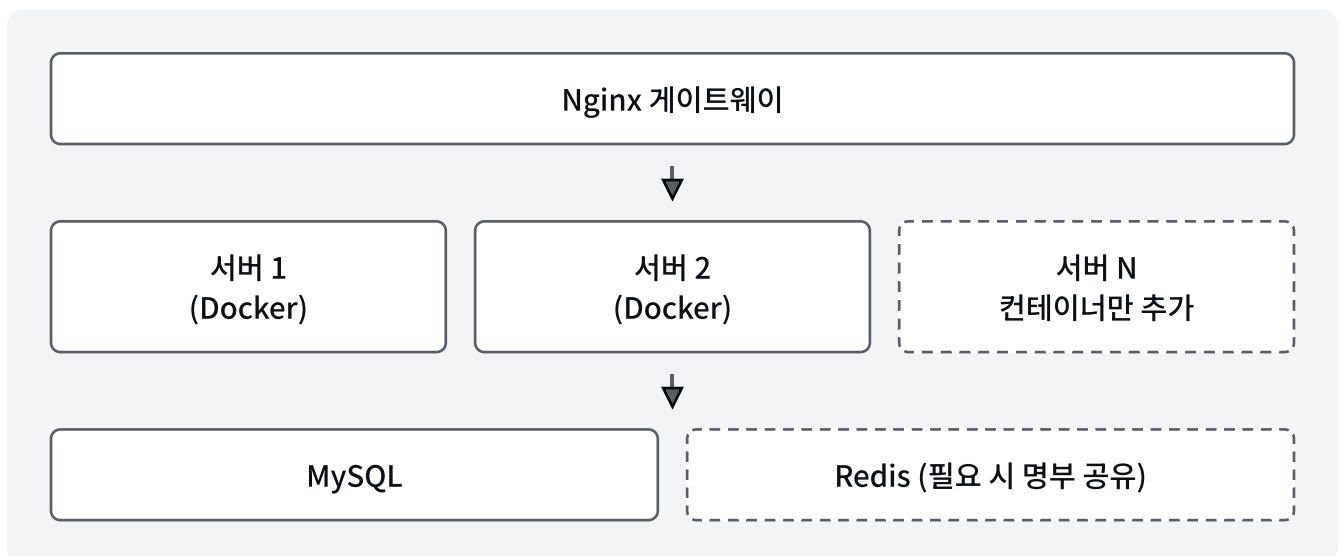
인프라

서버를 Docker 컨테이너로 묶어 같은 이미지를 어디서든 띄울 수 있게 했습니다. Nginx 게이트웨이 뒤에 컨테이너를 추가하면 서버가 늘어나고 배포는 GitHub Actions와 Docker Hub로 자동화돼 있어 몇 대가 되든 절차가 같습니다. 스폴 대조처럼 무거운 배치는 API 서버와 다른 프로세스로 떼어내 결제와 출력 경로가 영향을 받지 않게 했습니다.

코드

서버가 늘어도 고칠 코드가 없도록 설계했습니다. 로그인은 JWT라 어느 서버가 받아도 되고 결제 유일성은 DB가 보장합니다. 스케줄러는 DB 락으로 한 대에서만 실행되고 매장 연결 명부는 설정한 줄로 Redis 공유로 바꿀 수 있습니다.

서버를 늘리는 일이 이렇게 단순해집니다



서버가 몇 대가 되어도 안전한 이유

무상태 인증

JWT라 어떤 서버가 받아도 같은 결과

유일성은 DB

먹등키와 유니크 제약은 서버 수와 무관

스케줄러 DB 락

여러 대가 떠도 정리 작업은 한 번만

명부 공유 준비

설정 한 줄로 Redis 전환 가능

중설 0회

서버 한 대로 소화

코드 수정 0줄

서버 추가에 필요한 변경

하루 200명

개강 후 신규 가입 추세

커머스

주문과 결제와 재고가 한 번에 맞아떨어져야 하는 도메인이라 관심을 갖고 개인 프로젝트로 직접 설계했습니다.

2026.01 ~ 2026.03 · 개인 프로젝트, 1인 개발 · Java, Spring Boot, Redis, MySQL, k6

- 01 선착순 쿠폰과 재고의 동시성 제어
- 02 대기열과 이탈한 주문의 자동 복구

선착순 이벤트에서 재고가 새지 않도록 동시성 제어를 직접 설계했습니다

개인 프로젝트, 1인 개발 · Java, Spring Boot, JPA, MySQL, Redis, Docker, k6 · github.com/NohYeongO/zerofive-store

커머스는 주문과 결제와 재고가 한 번에 맞아떨어져야 하는 도메인이라 관심을 갖고 개인 프로젝트로 만들었습니다. Gradle 멀티모듈로 계정, 상품, 장바구니, 쿠폰, 주문, 결제를 나누고 모듈 사이는 Port 인터페이스로만 의존하게 해 결합도를 낮췄습니다.

문제

선착순 쿠폰과 재고 차감은 요청이 한꺼번에 몰립니다. 재고를 확인하고 차감하는 사이에 다른 요청이 끼어들면 재고보다 많이 발급되거나 한 사람이 여러 번 받는 일이 생깁니다.

설계

쿠폰은 Redis 멍등기로 이미 받은 사람을 먼저 걸러내고 재고 차감은 Lua 스크립트로 확인과 감소를 한 덩어리로 처리해 음수가 되면 즉시 되돌립니다. 이 둘 사이 틈으로 들어온 동시 요청은 MySQL 유니크 제약이 최종적으로 막습니다.

설계

상품 재고는 SELECT FOR UPDATE 비관적 락으로 잠급니다. 한 주문에 여러 상품이 담기면 상품 ID 순서로 정렬한 뒤 잠가 데드락을 예방했습니다.

쿠폰 한 장이 두 번 나가지 않게 막는 순서



Redis 검사와 DB 저장 사이에는 여전히 틈이 있습니다. 그 틈으로 들어온 요청은 유니크 제약에 걸리고 그 때 차감했던 재고를 되돌립니다. 유일성을 코드가 아니라 제약에 맡긴 것이 핵심입니다.

여러 상품을 한 번에 주문할 때



모든 요청이 같은 순서로 잠그면 서로를 기다리다 멈추는 데드락이 생기지 않습니다.

초과 발급 0건

재고 200장에 1,100명 동시
요청

중복 발급 0건

재요청 100건 전부 차단

데드락 0건

재고 50개에 100명 동시 주문

몰리는 요청은 대기열로 세우고 중간에 떠난 주문은 스스로 되돌립니다

선착순 대기열

문제

이벤트가 열리면 순간적으로 요청이 몰립니다. 들어온 순서는 지키면서 뒤에 온 요청이 발급 로직을 그대로 때리지 않게 해야 했습니다.

설계

Redis ZSET에 요청을 순서대로 쌓고 등록은 Lua 스크립트로 한 번에 처리합니다. 1초마다 도는 스케줄러가 앞에서부터 입장시켜 정해진 수만 발급으로 넘어갑니다.



주문 세션과 자동 복구

문제

주문을 시작하면 재고를 미리 잡아 둡니다. 손님이 결제하지 않고 창을 닫으면 그 재고가 묶인 채로 남아 팔 수 없게 됩니다.

설계

주문마다 세션을 만들고 화면이 10초마다 살아 있다고 알립니다. 30초마다 도는 스케줄러가 1분 넘게 소식이 없는 세션을 찾아 재고를 되돌리고 세션을 지웁니다.



사람이 챙기지 않아도 잡혀 있던 재고가 제자리로 돌아옵니다.

어떻게 확인했나

k6로 부하를 주고 InfluxDB와 Grafana로 응답 시간과 처리량을 봤습니다. 재고 200장에 1,100명이 몰린 상황에서 발급 200건, 중복 100건 차단, 품질 800건으로 의도한 값과 정확히 일치했고 발급 p95는 1.6초였습니다. 재고 50개에 100명이 동시 주문한 경우도 성공 50건에 데드락 0건이었습니다. 테스트는 Testcontainers로 실제 MySQL과 Redis를 띄워 검증했습니다.

1초

대기열 입장 주기

30초

만료 세션 스캔 주기

p95 1.6초

1,100명 동시 발급 요청 기준

FryDay

기획, 디자인, 프론트, 서버 6인이 함께 스토어에 출시하고
매주 회의하며 운영하고 있는 서비스입니다.

2025.11 ~ 운영 중 · 백엔드 2인 중 1 · Java, Spring Boot, MySQL, Redis, FCM, OCI

- 01 역할 분담과 팀이 일하는 방식
- 02 푸시 알림의 동시 수정과 중복 발송
- 03 월간 리포트의 정합성과 응답 속도

6인 팀으로 스토어에 출시하고, 매주 회의하며 함께 운영하는 서비스입니다

2025.11 ~ 운영 중 · 백엔드 2인 중 1 · Java, Spring Boot, JPA, MySQL, Redis, FCM, OCI · App Store, Google Play 출시

FryDay는 "할 일을 튀긴다"는 컨셉의 투두 앱입니다. 기획 1, 디자인 1, 프론트 2, 서버 2로 시작해 앱스토어와 플레이스토어에 출시했고, 지금도 매주 회의로 업데이트를 이어가고 있습니다. 서버 두 명이 도메인을 나눠 맡았고, 저는 회원과 알림, 월간 리포트를 책임졌습니다.

서버 둘이 이렇게 나눴습니다

내가 맡은 것

- 소셜 로그인 3종과 JWT 인증
- 탈퇴, 7일 재가입 제한, 자동 파기
- 푸시 알림 전체 (발송, 재시도)
- 월간 리포트, 전체 공지 발송

동료가 맡은 것

- 투두 작성, 정렬, 복사
- 카테고리, 캘린더
- 반복 일정 자동 생성
- 앱 캐릭터, 데일리 리워드
- 서버 모니터링 대시보드

일하는 방식



기획과 디자인, 프론트, 서버가 매주 모여 안건을 공유하고 한 주 단위로 계획을 세워 개발과 테스트를 진행합니다. 출시 후에는 사용자 피드백을 받아 다음 스프린트에 반영합니다. 코드는 브랜치와 PR 리뷰를 거쳐 병합합니다.

iOS · Android
스토어 동시 출시

매주 스프린트
회의부터 테스트까지

운영 중
사용자 피드백 반영

2인 서버
도메인 분담 협업

푸시 알림에서 만난 두 가지 문제, 동시 수정은 락으로 중복 발송은 트랜잭션 분리로 풀었습니다

자연되는 조회와 시간대 오차, 탈퇴자 발송 같은 기본 문제를 정리한 뒤에도 구조적인 문제가 남아 있었습니다.

문제 1. 발송과 시간 변경이 같은 데이터를 동시에 고칩니다

문제

1분마다 도는 발송 스케줄러와 사용자의 알림 시간 변경이 같은 알림 데이터를 동시에 고칠 수 있었습니다.

해결

충돌이 자주 나지는 않을 상황이라 판단해 낙관적 락(@Version)을 적용했고 충돌이 나면 그쪽만 다시 시도하게 했습니다.

낙관적 락을 고른 이유

두 작업이 같은 데이터를 동시에 건드리는 일 자체가 드뭅니다. 드문 충돌 때문에 매번 잠그는 것은 과하다고 봤습니다.

다시 생각해 본 것

낙관적 락도 버전 컬럼과 재시도 로직이 필요합니다. 충돌이 거의 없다면 비관적 락으로 잠가도 기다림이 없으니 코드가 더 단순했을 수 있습니다.

문제 2. 발송 전체가 한 트랜잭션이라 오류 하나가 중복을 만듭니다

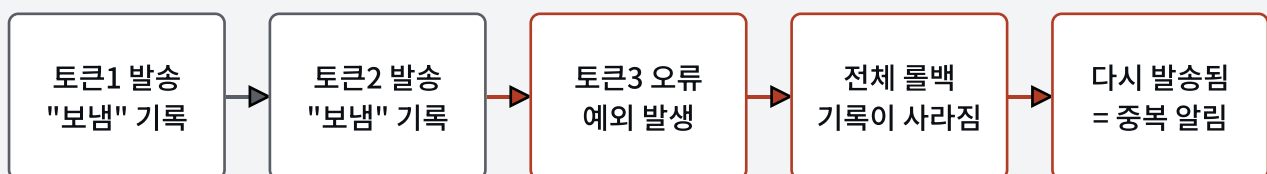
문제

4개월 뒤 기능을 넓히다 발견했습니다. 발송 전체가 한 트랜잭션이라 토큰 한 개의 오류가 그 실행분의 발송 기록을 모두 되돌렸고 이미 나간 푸시는 취소되지 않아 다음 회차에 중복 발송 될 수 있었습니다.

해결

발송 조회는 읽기 전용 트랜잭션으로 두고 토큰별로 예외를 격리했습니다. 실패한 토큰을 비활성화 하는 쓰기는 별도 트랜잭션(REQUIRES_NEW)으로 분리해 성공 기록이 함께 되돌아가지 않게 했습니다. 이 분리 설정이 같은 클래스 호출이라 동작하지 않던 문제도 함께 고쳤습니다.

BEFORE. 발송 전체가 한 트랜잭션



이미 나간 푸시는 취소되지 않는데 DB 기록만 사라져서 같은 알림이 폰에 두 번 도착할 수 있었습니다.

AFTER. 발송은 트랜잭션 밖 + 토큰별 격리



미리 만들어 두던 월간 리포트를 즉시 계산으로 바꾸고 비동기 조회로 속도를 지켰습니다

문제

월 스케줄러로 리포트를 미리 만들어 두고 조회하면 바로 내려주는 방식이었습니다. 그런데 사용자가 지난 기록을 고치면 미리 만든 값과 실재가 어긋났고 변경이 생길 때마다 다시 맞춰 주는 일이 쉽지 않았습니다.

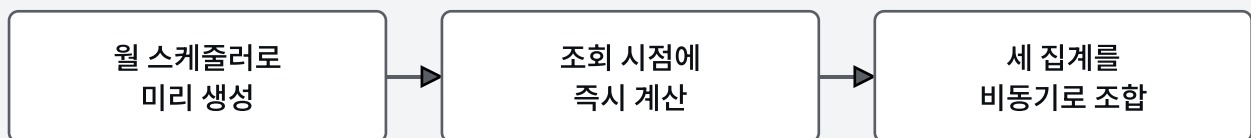
해결

미리 만든 데이터를 버리고 조회하는 시점에 즉시 계산하도록 바꿨습니다. 원본에서 바로 집계하기 때문에 기록을 언제 고쳐도 리포트는 항상 실재와 같습니다.

개선

즉시 계산은 세 가지 집계를 동기로 차례차례 조회해 느려졌습니다. 세 조회가 서로 독립임을 확인하고 `CompletableFuture`로 동시에 실행해 조합하는 방식으로 수정했습니다. 검증은 실제 MySQL을 띄우는 테스트(`Testcontainers`)를 팀에 처음 도입해 진행했습니다.

리포트가 만들어지는 방식의 변화



정합성의 원천을 원본 데이터 하나로 통일하고 그 대가로 생긴 조회 비용은 병렬 조회로 되찾았습니다.

3개 집계 데이터를 가져오는 방법의 차이

BEFORE. 동기 순차 조회 - 셋을 차례로 기다립니다



AFTER. 비동기 동시 조회 - 셋이 함께 출발합니다



`CompletableFuture`로 셋을 동시에 실행해 가장 오래 걸리는 조회만큼만 기다립니다. 공용 풀 대신 전용 스레드 풀에서 돌려 다른 요청에 영향을 주지 않게 했습니다.

항상 최신

원본에서 그때그때 계산

셋이 동시 출발

가장 느린 조회만큼만 대기

실제 DB로 검증

`Testcontainers` 팀 첫 도입